Modèles Algorithmiques Fondamentaux et Mécanismes d'Accès aux Données

Thème principal : Conception d'algorithmes canoniques sur structures à accès direct et séquentiel en C++

Synthèse exécutive : Ce cours pose le cadre méthodologique fondamental du développement logiciel en BUT 1 Informatique. Avant tout codage en C++, le concepteur doit impérativement identifier le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours détaille la mise en œuvre de trois modèles algorithmiques universels sur structures à accès direct (tableaux, chaînes) : le parcours complet systématique, le parcours complet conditionné et la recherche de première occurrence. L'accent est mis sur la rigueur de la gestion des bornes (indices 0 à N-1 pour éviter le dépassement de mémoire 'Out of Bounds'), l'adoption du principe de pessimisme algorithmique et l'obligation stricte d'intégrer une double condition d'arrêt dans toute recherche.

1. Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

L'étape préliminaire absolue de tout problème algorithmique consiste à analyser le contrat d'interface offert par la structure de données sous-jacente. Si la structure permet un accès direct (comme un tableau statique ou une chaîne de caractères en C++), le processeur calcule l'adresse mémoire de n'importe quel élément à l'indice 'i' de manière instantanée. À l'inverse, un flux séquentiel (tel qu'un fichier texte ou un fichier CSV lu ligne par ligne) ne met à disposition que trois primitives : ouvrir/se positionner au début, lire le composant suivant, et tester la fin de fichier. Tenter d'accéder directement à un élément distant par un indice T[i] sur un fichier séquentiel constitue une faute logique majeure qui invalide intégralement l'algorithme.

- Structure de données : Collection organisée régroupant des éléments de même type.
- Accès direct : Capacité d'atteindre instantanément n'importe quel élément via un indice T[i] en temps constant O(1).
- Accès séquentiel : Contrainte imposant de se positionner au début et de progresser pas à pas (début, suivant, fin) sans saut direct possible.
- Accès dynamique : Structure hybride proposant simultanément des primitives d'accès séquentiel et d'indexation directe.
- Incompatibilité structurelle : Interdiction absolue d'utiliser la notation par indice T[i] sur un flux séquentiel.

Cas concret enseignant : *Le professeur souligne une erreur fréquente aux examens : utiliser un indice T[i] pour traiter une ligne d'un fichier texte ou CSV brut. Le système d'exploitation ne fournissant qu'un curseur d'avancement pas à pas, cette syntaxe conduit directement à la note zéro.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Le modèle du parcours complet systématique est mobilisé lorsqu'un traitement doit s'appliquer de façon inconditionnelle à la totalité des éléments compris entre la première et la dernière position. Les cas d'usage classiques incluent l'affichage intégral d'un tableau, la remise à zéro de ses cases ou la conversion en majuscules d'une chaîne. Puisque le nombre de répétitions est parfaitement déterminé avant d'entrer dans la boucle (N itérations), la structure logique adaptée est la boucle bornée 'Pour'. En C++, la première case d'un tableau de taille N étant située à l'indice 0, la dernière case valide est strictement à l'indice N-1.

- Exhaustivité : Traitement appliqué à 100% des éléments d'une plage donnée.
- Structure itérative : Utilisation obligatoire d'une boucle bornée Pour (nombre d'itérations connu a priori).
- Bornes d'indices : De 0 à N-1 pour une structure de taille N indicée à partir de 0.
- Erreur 'Out of Bounds' : Dépassement de capacité provoqué par une tentative d'accès à l'indice N.

Cas concret enseignant : *Exemple d'amphi : 'Afficher les éléments d'un tableau de taille N'. L'enseignant rappelle que l'erreur type de débutant est d'écrire 'Pour i allant de 0 à N', ce qui déclenche un plantage à l'exécution (crash mémoire / segmentation fault) lors de la tentative de lecture de la case hors limites T[N].*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Ce second modèle conserve le principe d'un examen systématique de l'ensemble de la collection, mais conditionne l'exécution du traitement à la validation d'un test logique. Il est utilisé pour le filtrage de données (ex: afficher les nombres pairs), l'accumulation sélective (ex: calculer la somme des entiers positifs) ou le décompte d'occurrences. Une attention toute particulière doit être portée aux conditions comparant des éléments voisins : si le test s'appuie sur T[i] et T[i-1], la boucle doit impérativement débuter à i=1 (et non 0) sous peine de tenter une lecture invalide sur T[-1].

- Parcours exhaustif : Tous les éléments du tableau sont examinés du début à la fin.
- Traitement sélectif : L'action n'est déclenchée que si le composant courant satisfait un prédicat logique.
- Imbrication structurelle : Utilisation d'une structure Si...Alors à l'intérieur d'une boucle bornée Pour.
- Ajustement des bornes : Obligation de modifier les bornes de la boucle lorsque la condition compare des éléments adjacents (i-1 ou i+1).

Cas concret enseignant : *Analogie de la chaîne de montage : 'Chaque pièce défile systématiquement devant le contrôleur qualité (parcours complet d'inspection), mais ce dernier n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut (traitement conditionné).'*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux parcours complets, l'algorithme de recherche de première occurrence vise à stopper l'itération au plus tôt dès que l'élément cible est identifié. Le concepteur doit obligatoirement fonder sa logique sur le principe du pessimisme algorithmique : postuler initialement que la cible n'existe pas dans le tableau. De plus, une recherche algorithmiquement correcte DOIT comporter deux motifs de sortie de boucle : soit la réussite (cible localisée), soit l'échec (fin du tableau atteinte). L'absence de l'une de ces deux conditions entraîne un risque soit de boucle infinie, soit d'accès mémoire invalide.

- Pessimisme algorithmique : Initialisation d'un drapeau booléen 'trouvé' à Faux avant l'exploration.
- Boucle non bornée : Emploi d'un Tant que car le nombre d'itérations nécessaire n'est pas connu à l'avance.
- Double condition d'arrêt : L'itération s'arrête si l'élément est trouvé OU si la collection a été intégralement parcourue sans succès.
- Variante 1 (Rupture immédiate) : Interruption nette dés la détection ; conserve l'indice i exactement sur l'élément trouvé.
- Variante 2 (Tant que global) : Test unique en tête de boucle ; structurelle et standard, mais incrémente l'indice d'un pas en sortie.

Cas concret enseignant : *Devise du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que la valeur n'existe pas, et ne passez votre drapeau à Vrai qu'au moment précis où vous la touchez.' Une recherche présentée en partiel sans double condition d'arrêt est systématiquement sanctionnée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui. Une structure à accès dynamique offrant à la fois les fonctionnalités d'un accès séquentiel et l'indexation directe, il est tout à fait valide d'y effectuer un parcours complet systématique au moyen d'une boucle bornée.

Question étudiant : Pourquoi présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La variante 1 (avec interruption immédiate) permet de conserver l'indice 'i' exactement positionné sur l'élément trouvé sans instruction supplémentaire. La variante 2 (Tant que avec test combiné en tête) offre une écriture plus homogène et évite l'utilisation d'instructions de rupture brutale, mais nécessite de réajuster l'indice ou le test après la sortie.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut impérativement réajuster les bornes de la boucle 'Pour'. Si vous comparez T[i] à T[i-1], faites démarrer la boucle à i=1. Si vous comparez T[i] à T[i+1], faites s'arrêter la boucle à N-2 pour éviter un dépassement de mémoire.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur (ex: trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il faut utiliser le deuxième modèle (Parcours complet avec traitement conditionnel), car il est nécessaire d'examiner l'intégralité du tableau de l'indice 0 à N-1 sans s'arrêter au premier résultat trouvé.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme C++ complet respectant les normes de codage de l'IUT de Bayonne (cartouche, tripartition, syntaxe Allman) permettant de saisir un tableau d'entiers de taille N, puis d'afficher uniquement les valeurs strictement positives selon le modèle du parcours complet conditionné.

Démarche de résolution :
1. Identification de la structure et du modèle : On utilise un tableau à accès direct d'entiers. Le problème exige d'examiner chaque élément de la borne 0 à nbElements - 1 (parcours complet) et d'appliquer une action (affichage) uniquement si l'élément vérifie 'tab[i] > 0' (traitement conditionné). 2. Gestion des bornes : L'indice i varie strictement de 0 à nbElements - 1. La taille maximale réservée en mémoire est sécurisée par la constante TAILLE_MAX. 3. Respect de la norme C++ IUT Anglet : Structure de code séparée en trois blocs (CONSTANTES, VARIABLES, TRAITEMENTS), style d'accolades Allman et nommage explicite.

```cpp
/*
  En-tête / Cartouche d'en-tête
  Nom du fichier : exercice_parcours_conditionnel.cpp
  Auteur : Matéi de Saint Léger
  Date : 14/11/2026
  Objectif : Afficher les valeurs strictement positives d'un tableau d'entiers
*/

#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES
    const int TAILLE_MAX = 100;

    // VARIABLES
    int tab[TAILLE_MAX];
    int nbElements;
    int i;

    // TRAITEMENTS
    // Saisie du nombre d'éléments effectifs
    cout << "Entrez le nombre d'elements (max 100) : ";
    cin >> nbElements;

    // Saisie des éléments du tableau
    for (i = 0; i < nbElements; i++)
    {
        cout << "Elements [" << i << "] : ";
        cin >> tab[i];
    }

    // Parcours complet avec traitement conditionnel
    cout << "\n--- Liste des nombres strictement positifs ---" << endl;
    for (i = 0; i < nbElements; i++)
    {
        if (tab[i] > 0)
        {
            cout << "Valeur positive trouvee a l'indice " << i << " : " << tab[i] << endl;
        }
    }

    return 0;
}
```

Résultat : Le programme parcourt sans défaillance le tableau saisi, ignore les entiers négatifs ou nuls, et affiche l'indice ainsi que la valeur de chaque entier strictement positif, sans risquer de débordement mémoire 'Out of Bounds'.

■ Auto-Évaluation Flash d'Examen

Q : Quel risque majeur prend-on si l'on écrit une boucle 'for (int i = 0; i <= N; i++)' pour parcourir un tableau de N éléments ?

R : Une erreur 'Out of Bounds' (dépassement d'indice mémoire) lors de l'accès à tab[N], car les cases valides d'un tableau de taille N sont indicées de 0 à N-1.

Q : Quelles sont les deux conditions d'arrêt obligatoires dans un algorithme de recherche de première occurrence ?

R : 1. La détection de l'élément recherché (succès). 2. L'atteinte de la fin du tableau sans l'avoir trouvé (épuisement de la structure).

Q : Pourquoi qualifie-t-on de 'pessimiste' la démarche d'initialisation de la recherche de première occurrence ?

R : Parce qu'on initialise systématiquement le drapeau booléen 'trouvé' à Faux avant d'entamer la boucle, en supposant par défaut que l'élément n'existe pas.